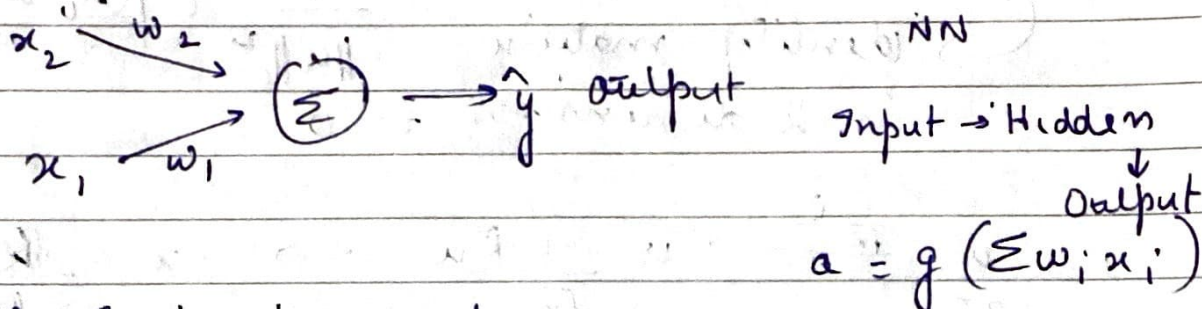


DL Gen AI - Week-2

L1 Universal Approximation Theorem

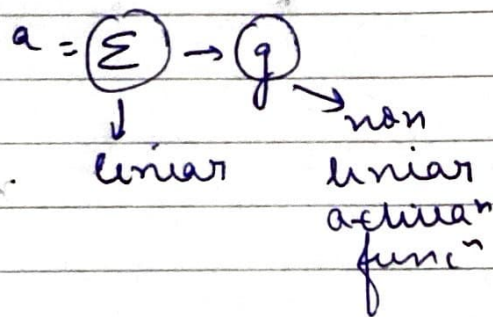
- LR recap



* A LR is weighted

sum of input features.

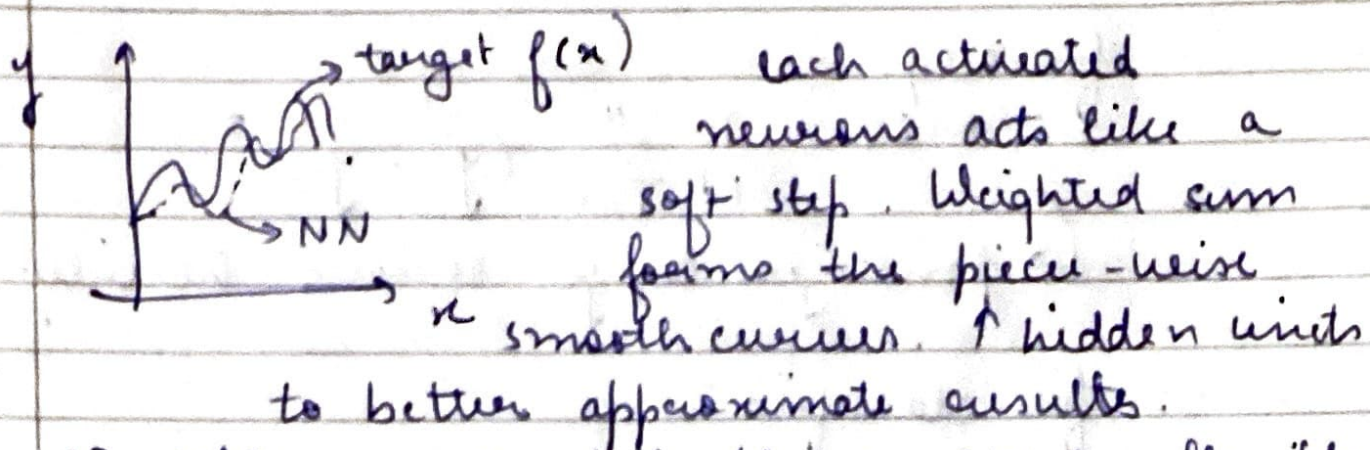
$$\hat{y} = w_0 + w_1 x = \sum w_i x_i$$



- UAT $\rightarrow$ A feedforward network with a single hidden layer & a non-linear, bounded, continuous activation function can approximate any continuous function on a compact domain to arbitrary accuracy.

$\rightarrow$ capacity $\rightarrow$ existence of params for better results

$\rightarrow$ not a training guarantee, depth is not req., but it improves accuracy.



- LR captures max hyperplanes. NN has flexible funcⁿ families. VBT says about learnability, but not on generalizaⁿ.

L2 Forward Pass & Loss Functions

- Process of taking input x & feed it through network layer by layer, reach final predictⁿ $\hat{y}$.

$$x = a^{[0]} \rightarrow z' = w' a^0 + b' \rightarrow g \dots \rightarrow y = a^{[L]}$$

every layer has -

1. linear combineⁿ - weighted sum of all neurons in the layer.
2. non-linear activaⁿ - apply activaⁿ funcⁿ element-wise.

final $a^{[L]}$ is the predictⁿ $\hat{y}$.

- A loss funcⁿ $L(y, \hat{y})$, returns error. We try to minimize this.

- for reg.

$$MSE = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2$$

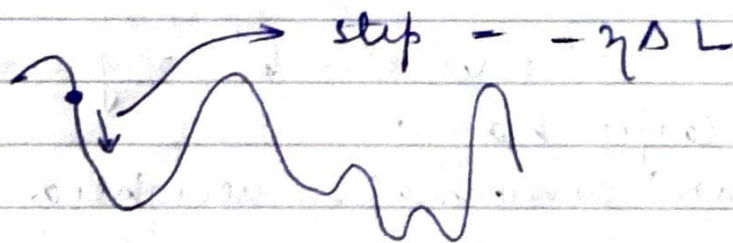
↳ penalizes large errors heavily.

- for classificaⁿ

$$\text{Cross entropy; } L = - \sum_{c=1}^C y_c \log(\hat{y}_c)$$

↳ penalizes the model if it is wrong & yet confident.
predicted probab.

L3 Gradient descent



direcⁿ of steepest ascent is gradient (∇L)

so we take steps in opp. direcⁿ

- if $\Delta L \rightarrow \eta$ (step) is too small, it will take forever, if it is too big, it overshoots the valley.

$$\boxed{\theta_{\text{new}} = \theta_{\text{old}} - \eta \Delta L(\theta_{\text{old}})}$$

(at each step, we move in the direcⁿ of steepest ascent)

- global minima $\rightarrow$ lowest pt.; optimal soluⁿ
- local minima $\rightarrow$ valley that traps the algo., prevents it to reach global minima

$$\hat{y} = mx + c, \quad \theta = \{m, c\}$$
$$L(m, c) = \frac{1}{N} \sum (\hat{y}_i - y_i)^2$$

update

$$m = m - \eta \frac{\partial L}{\partial m}$$

$$c = c - \eta \frac{\partial L}{\partial c}$$

- refer to lectures slides $\rightarrow$ for diff.
Batch GD, Stochastic GD, Mini-batch GD.
entire dataset $\rightarrow$ single pt. $\rightarrow$ mini-batch of data.
this is preferred in most DL applications
efficient in terms of speed, convergence,
& resources required for computaⁿ.

L4

Customer Churn Predictⁿ: Overview

- customer churn problem statement
- we analyze, engineer, handling $\rightarrow$ pre-build, evaluate, interpret, design

- business fact — retaining customer is much cost-effective than acquiring new ones.
- target is $\rightarrow$ binary $\rightarrow \{0, 1\}$
whether or not $\swarrow$
the customer will churn next month.
- we have several features for services, usage patterns, demographics, etc.
- metrics $\rightarrow$ precision, recall, F1, AUC-ROC.

L5

Churn: Data Processing

- Kaggle churn dataset.
- imports, reproducibility, data loader, put it in df
- do data cleaning $\rightarrow$ outliers, missing values
- EDA $\rightarrow$ to understand & visualize patterns, check feature correlation.
- FE $\rightarrow$ normalization, create bins, remove bad fields, do class balancing.

L6

Neural Network Package Imports

- _/_/_
- import torch.nn as nn
 - optim.lr_scheduler
 - metrics
 - optim
 - ← adam as optimizer
 - classification_report,
 - roc_auc_score,
 - confusion_matrix
 - nn.functional as F
- Reduce LROn Plateau

L7 Neural Network Implementation

- class ChurnPredictor
 - def __init__(self, input, hidden, dropout)
 - linear layer
 - ↓
 - batch normalization
 - ↓
 - activation (ReLU) $\leftarrow \max(0, x)$
 - ↓
 - dropout
 - ← applying sigmoid non-linearity
 - getting single logit to get probabilities
- output layer

nn.Sequential $\Rightarrow$ becomes an independent module
finally, nets are initialized

FC $\rightarrow$ Fully connected

L8

Churn: Weight Initialization

def - initialize $\rightarrow$ make a custom one.

if isinstance (module, nn.Linear)

popular
choice
for
FC layers

// use Xavier/Glorot initialize
set all bias to 0.

$\hookrightarrow$ this avoids any arbitrary
offset in the beginning

forward pass

$\hookrightarrow$ self.network $\Rightarrow$ it passes
all my layers into the
network

L9

Churn: Training & Validation Setup

- input size $\leftarrow$ no. of features
create the model $\Rightarrow$ call class
Churn Predictor
move to GPU

p.minrel() for p in parameters()

$\hookrightarrow$ gives the total no. of
parameters for this model.

define criterion

↳ BCEWithLogitsLoss

optimizer $\Rightarrow$ optim. Adam

Scheduler $\Rightarrow$ Reduce

LROnPlateau.

sigmoids +
binary
cross entropy
loss

it handles
class
imbalance

setup the train / test split into

X-train, y-train, X-val, y-val

↳ you have to stratify so that you
get equal proportion of 0 & 1 classes.

- now we have to define the training
funcⁿ $\Rightarrow$ we will do entire batch
(since dataset is small).

model.train()

// zero out all your gradients

// squeeze it (forward pass)

// calculate the loss

// do backpropagaⁿ

// update your weights by using For.

$\Rightarrow$ return your loss

- similarly write funcⁿ for validating
your dataset.

model.eval()

// apply sigmoid on your outputs
to get predictions in probab.

from predictions $\Rightarrow$ go and select the predicted class.

// if here your accuracy isn't good, first go and play with the learning rate

L10

Chuen: Training Loop Implementation

number of epochs

keep the best validation loss

keep the training & validation losses

better to keep an early stopping

Criteria based on a certain threshold

train - loss $\Rightarrow$ everything (all steps)

similarly,

val - loss

Scheduler. step()

$\rightarrow$ to handle the learning rate based on the

keep all the metrics validation loss to later judge how our model has performed.

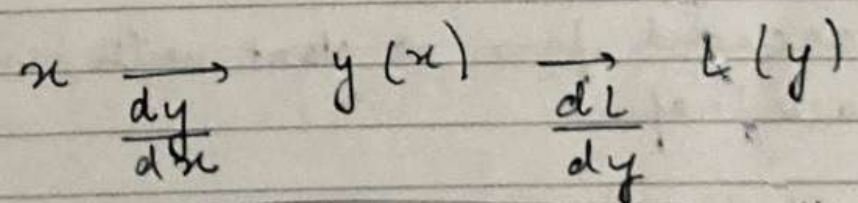
L11 Churn: Model Training

- based on initializaⁿ, wait for the convergence.
- never try to visualize them -
 1. training / val loss per epochs
 2. accuracy over epoch
 3. learning rate " "
 4. overfitting check
- play with learning rate value.

L12 Backpropagaⁿ: Simple Scalar Chain

- ↳ it helps us to compute the gradients
- how will the small Δ in param. affect the final loss.

$L(y(x))$, compute $\frac{dL}{dx}$



$$\boxed{\frac{dL}{dx} = \frac{dL}{dy} \times \frac{dy}{dx}}$$

Chain rule

influence of x on L is product of the local influences along the path

$$\left. \begin{array}{l} L \text{ as a function of } y \\ y \text{ as a function of } x \end{array} \right\} \text{ we get } \frac{dL}{dx}$$

- for vector func^{ns}, $y = f(x)$ & $L = g(y)$, the derivatives become Jacobian matrices & chain rule become matrix.

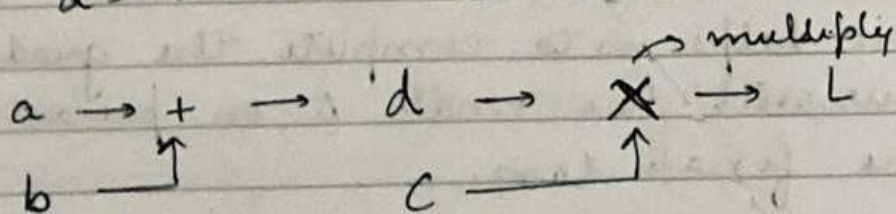
$$\boxed{\nabla_x L = (J_y L') (J_x y)}$$

- Jacobian matrix $\rightarrow$ contains all first-order partial derivatives of a vector valued funcⁿ. It represents func^{ns} non-linear approximaⁿ & sensitivity towards output.

$$L = (a+b) \cdot c$$

$$d = a+b$$

$$\& \quad L = d \cdot c$$



$$L = (a+b) \cdot c$$

Forward pass $\rightarrow$ start with input values
back pass.

$$L = (a+b) \cdot c, \quad a=2, b=3, c=6$$

$$\frac{dL}{dL} = 1$$

$$\frac{dL}{dc} = \frac{dL}{dL} \times \frac{dL}{dc} = 1 \cdot d = 5$$

$$\frac{dL}{da} = \frac{dL}{dL} \times \frac{dL}{da} = 1 \cdot c = 6$$

L13

Back Prop: MLP $\rightarrow$ multi-layered perception

- goal is to calculate the gradient of loss L w.r.t every wt. $W_{ij}^{[l]}$ & bias $b_i^{[l]}$

$$a^{[l-1]} \xrightarrow{W^{[l]}, b^{[l]}} z^{[l]} \xrightarrow{g(\cdot)} a^{[l]} \rightarrow L$$

for wts $W^{[l]}$, $\frac{\partial L}{\partial W^{[l]}}$

$$\left[\frac{\partial L}{\partial W^{[l]}} = \underbrace{\frac{\partial L}{\partial a^{[l]}}}_{\text{upstream}} \times \underbrace{\frac{\partial a^{[l]}}{\partial z^{[l]}}}_{\text{activation}} \times \underbrace{\frac{\partial z^{[l]}}{\partial W^{[l]}}}_{\text{local}} \right]$$

- derivation of wt. update rule.

$$1. \quad s^l = \frac{\partial L}{\partial z^{(l)}}$$

$$s^l = \frac{\partial L}{\partial a^{(l)}} \times \frac{\partial a^{(l)}}{\partial z^{(l)}} \quad \text{element-wise multiplication}$$

$$\frac{\partial L}{\partial a^{(l)}} = (W^{[l+1]})^T s^{[l+1]}$$

$$\Rightarrow \boxed{s^{[l]} = \left((W^{[l+1]})^T s^{[l+1]} \right) \odot g'(z^{[l]})}$$

for the output layer, it simplifies to -

$$s^{[l]} = \nabla a^L \odot g'(z^{[l]})$$

2. now we have to calculate the gradients for nets & biases.

$$\boxed{\frac{\partial L}{\partial w^{(l)}} = \delta^{(l)} (a^{(l-1)})^T}$$

$$\boxed{\frac{\partial L}{\partial b^{(l)}} = \delta^{(l)}}$$

now, these gradients can be used to update our parameters.

L14 XOR Problem: Intro

- XOR $\Rightarrow$ not linearly separable
 $\Downarrow$
 there is no single straight line which will put two classes separate.
 $\Downarrow$ here perceptron fails.

$$f(x_1, x_2) = w_1 x_1 + w_2 x_2 + b$$

$$\hat{y} = \sigma(f(x_1, x_2))$$

decision boundary $\Rightarrow f(x_1, x_2) = 0$
 $\hookrightarrow 0.5$

eqⁿ, if separates 2 classes is -

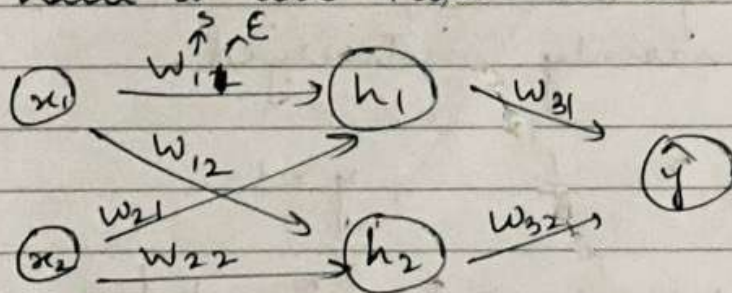
$$w_1 x_1 + w_2 x_2 + b = 0$$

for our model to solve XOR $\Rightarrow$

$$\begin{cases} (0, 0) = b \approx 0 \\ (0, 1) = w_2 + b \approx 1 \\ (1, 0) = w_1 + b \approx 1 \\ (1, 1) = w_1 + w_2 + b \approx 0 \end{cases}$$

but this can't be achieved even.

- We have to use non-linear activation



forward pass

$$z_1 = w_{11}x_1 + w_{21}x_2 + b \Rightarrow h_1 = \sigma(z_1)$$

$$z_2 = w_{12}x_1 + w_{22}x_2 + b \Rightarrow h_2 = \sigma(z_2)$$

$$z_3 = w_{31}h_1 + w_{32}h_2 + b \Rightarrow \hat{y} = \sigma(z_3)$$

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

$$L(y, \hat{y}) = -[y \log(y) + (1-y) \log(1-\hat{y})]$$

$\hookrightarrow$ binary cross entropy loss

L15XOR: Forward Pass

- Ex $x = [1, 0]^T$, $y = 1$

$$W_1 = \begin{pmatrix} W_{11} & W_{12} \\ W_{21} & W_{22} \end{pmatrix} = \begin{bmatrix} 1 & -1 \\ 1 & -1 \end{bmatrix}$$

$$b_1 = (b_1, b_2) = [0, 1]$$

$$W_2 = [W_{31}, W_{32}] = [2, -1]$$

$$b_2 = (b_3) = -1$$

remember, when neural network is actually trained, that time, this will be randomly initialized.

$$x_1 = 1, x_2 = 0, y = 1$$

$$z_1 = W_{11} x_1 + W_{21} x_2 + b_1 = 1$$

$$z_2 = 0$$

$$h_1 = \sigma(z_1) = 0.7$$

$$h_2 = \sigma(z_2) = 0.5$$

$$z_3 = -0.038, \hat{y} \approx +0.491$$

$$L = 0.712$$

$\frac{\partial L}{\partial W_{ij}} \Rightarrow$ we have to backpropagate now to get all wts W_{ij} using

$$W_{\text{new}} = W_{\text{old}} - \alpha \frac{\partial L}{\partial W_{\text{old}}}$$

L16 XOR: Backpropagation

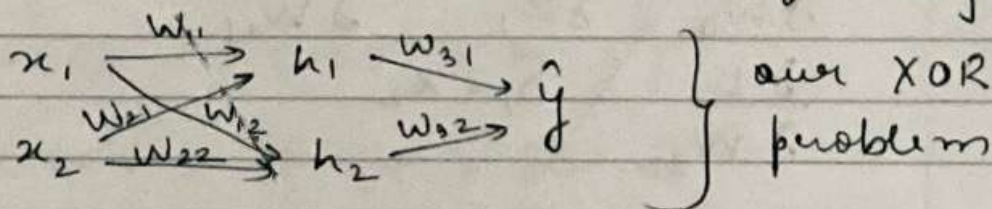
Recall $\delta^{(L)} = \frac{\partial L}{\partial a^{(L)}} \odot \frac{\partial a^{(L)}}{\partial z^{(L)}}$

$$(w^{(L+1)})^T \delta^{(L+1)}$$

$$\rightarrow g'(z^{(L)})$$

activation derivatives

error propagated back from next layer.



recall the formulas.

- input derivative

$$\rightarrow \sigma'(z) = \sigma(z) (1 - \sigma(z))$$

$$\rightarrow \frac{\partial L}{\partial \hat{y}} = \frac{\hat{y} - y}{\hat{y}(1 - \hat{y})}$$

- chain rule stuff.

→ calculate gradient backward.

$$\begin{aligned} \frac{\partial L}{\partial z_3} &= \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial z_3} \\ &= \frac{\hat{y} - y}{\hat{y}(1 - \hat{y})} \cdot (\hat{y}(1 - \hat{y})) \end{aligned}$$

$$\delta_3 = (\hat{y} - y)$$

$$\frac{\partial L}{\partial w_{31}} = \frac{\partial L}{\partial z_3} \cdot \frac{\partial z_3}{\partial w_{31}} = \delta_3 \cdot h_1$$

$$\frac{\partial L}{\partial w_{32}} = \delta_3 \cdot h_2$$

$$\frac{\delta L}{\delta z_1} = \frac{\delta L}{\delta z_3} \cdot \frac{\delta z_3}{\delta h_1} \cdot \frac{\delta h_1}{\delta z_1} = \delta_3 \cdot w_{31} \cdot \sigma'(z_1)$$

$$\frac{\delta L}{\delta z_2} = \delta_3 \cdot w_{32} \cdot \sigma'(z_2)$$

$$\frac{\partial L}{\partial w_{11}} = \frac{\partial L}{\partial z_1} \cdot \frac{\partial z_1}{\partial w_{11}} = \delta_1 \cdot x_1$$

$$\frac{\partial L}{\partial w_{22}} = \delta_1 \cdot x_2$$

now just plug in the mathematical values to get your gradients.
Remember, you can calculate these things (layer-by-layer), from outermost to innermost.

Now, as per the update rule.

$$w_{\text{new}} = w_{\text{old}} - \alpha \frac{\partial L}{\partial w}$$

learning rate

just get the new values.

- the hidden layer is able to transform the input data into a new representation which is linearly separable.

L17 Fashion MNIST Classification: Overview

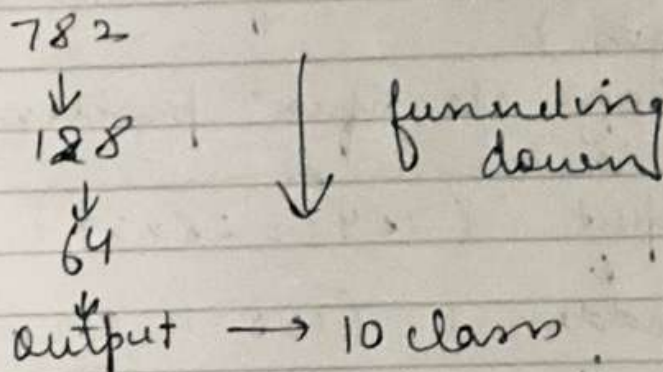
- build a MLP for MNIST (handwritten digits) ^{inspired by}
- image classification problems (CV intro)
- 10 classes
- input ($784 = 28 \times 28$) $\rightarrow$ flattened
 - $\downarrow$
 - hidden 1 (128 + ReLU)
 - $\downarrow$
 - hidden 2 (64 + ReLU)
 - $\downarrow$
 - output (10 classes + softmax)
- use dataloader to bring the dataset.
- specify the batch size to implement mini-batch.
- keep shuffle = True, so that the model does not learn the order of the data pts
- now move to data visualization & see how our dataset looks like.

L18 MNIST: MLP Implementation

- define new class FashionMLP

$28 \times 28 \Rightarrow$ flatten 1D vector

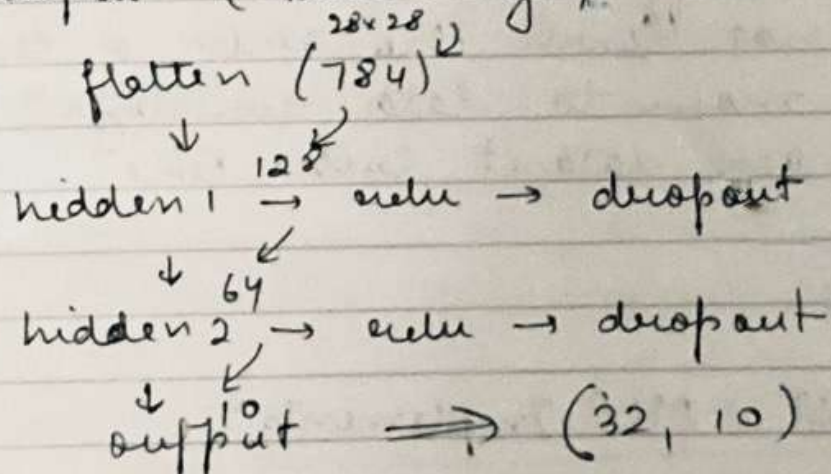
hidden layers helps you to learn the abstract features of the images.



- between layers, add ReLU activation. Then, apply dropout optionally.

L19 MNIST: Forward Pass

- input tensor (1, 28, 28)
output (batch-size, num-classes)



return the logits. $\Rightarrow$ apply softmax to get the probabilities
compile your model architecture now

L20 MNIST: Training Loop

- define the training config
 - learning rate
 - no. of epochs
 - weight decay $\Rightarrow$ L2 regularization to avoid overfits
- define your loss funcⁿ & optimizer
- set the funcⁿ for training loop.
 - ↳ then for every mini-batch the model gets trained.
 - after every batch, calculate the loss & accuracy. $\Rightarrow$ keep updating the weights & zeroing out the gradients.

L21 Validatⁿ Loop: MNIST

- define the model.eval() $\Rightarrow$ disables dropouts
 - ↳ disable the gradient computaⁿ so that we have time efficiency.
- keep all values $\Rightarrow$ it can later create confusion matrix to get recall, precision & F1.

L22 MNIST: Model Training

- now everything is defined, we can start our main training loop.
- iterate btw training & evalⁿ loop to check how the model gets better
- define early stopping as per the requirements.
- later, see using visualizaⁿ, how it looks.
- overfit = train - accuracy - test - accuracy. Keep a threshold to check whether overfit happens or not.
- just remember, by flattening you have forgotten all the spatial info. This will get solved when we use CNNs.